



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



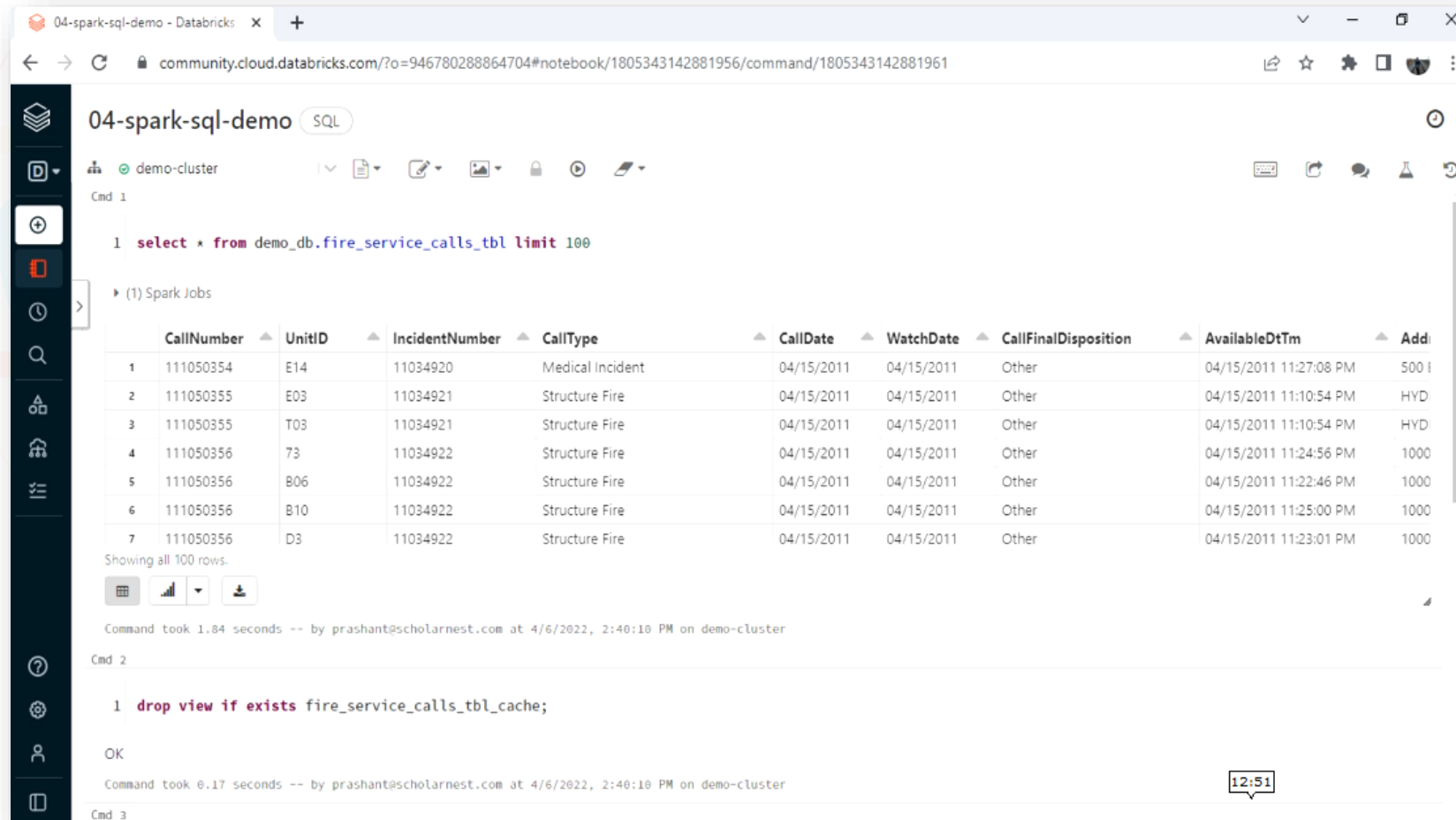
Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Working with Spark SQL

Go back to your notebook you created in the previous lecture. **(Reference: 04-spark-sql-demo.sql)**

We created a table and also cached it into the memory. Now we are ready to start writing some SQL expressions to answer the 10 micro project questions.



The screenshot shows a Databricks notebook interface with the title "04-spark-sql-demo" and a "SQL" tab. The notebook is running on a "demo-cluster". The first command (Cmd 1) is a SQL query: `1 select * from demo_db.fire_service_calls_tbl limit 100`. Below the command, it shows "(1) Spark Jobs" and a table of results. The table has 10 columns: CallNumber, UnitID, IncidentNumber, CallType, CallDate, WatchDate, CallFinalDisposition, AvailableDtTm, and Address. The results show 7 rows of data. Below the table, it says "Showing all 100 rows." and provides a download icon. The second command (Cmd 2) is a SQL query: `1 drop view if exists fire_service_calls_tbl_cache;`. Below the command, it says "OK" and provides a download icon. The third command (Cmd 3) is empty. The notebook interface includes a sidebar with various icons and a top navigation bar.

	CallNumber	UnitID	IncidentNumber	CallType	CallDate	WatchDate	CallFinalDisposition	AvailableDtTm	Address
1	111050354	E14	11034920	Medical Incident	04/15/2011	04/15/2011	Other	04/15/2011 11:27:08 PM	500 I
2	111050355	E03	11034921	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:10:54 PM	HYD
3	111050355	T03	11034921	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:10:54 PM	HYD
4	111050356	73	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:24:56 PM	1000
5	111050356	B06	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:22:46 PM	1000
6	111050356	B10	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:25:00 PM	1000
7	111050356	D3	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:23:01 PM	1000

Command took 1.84 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:40:10 PM on demo-cluster

Cmd 2

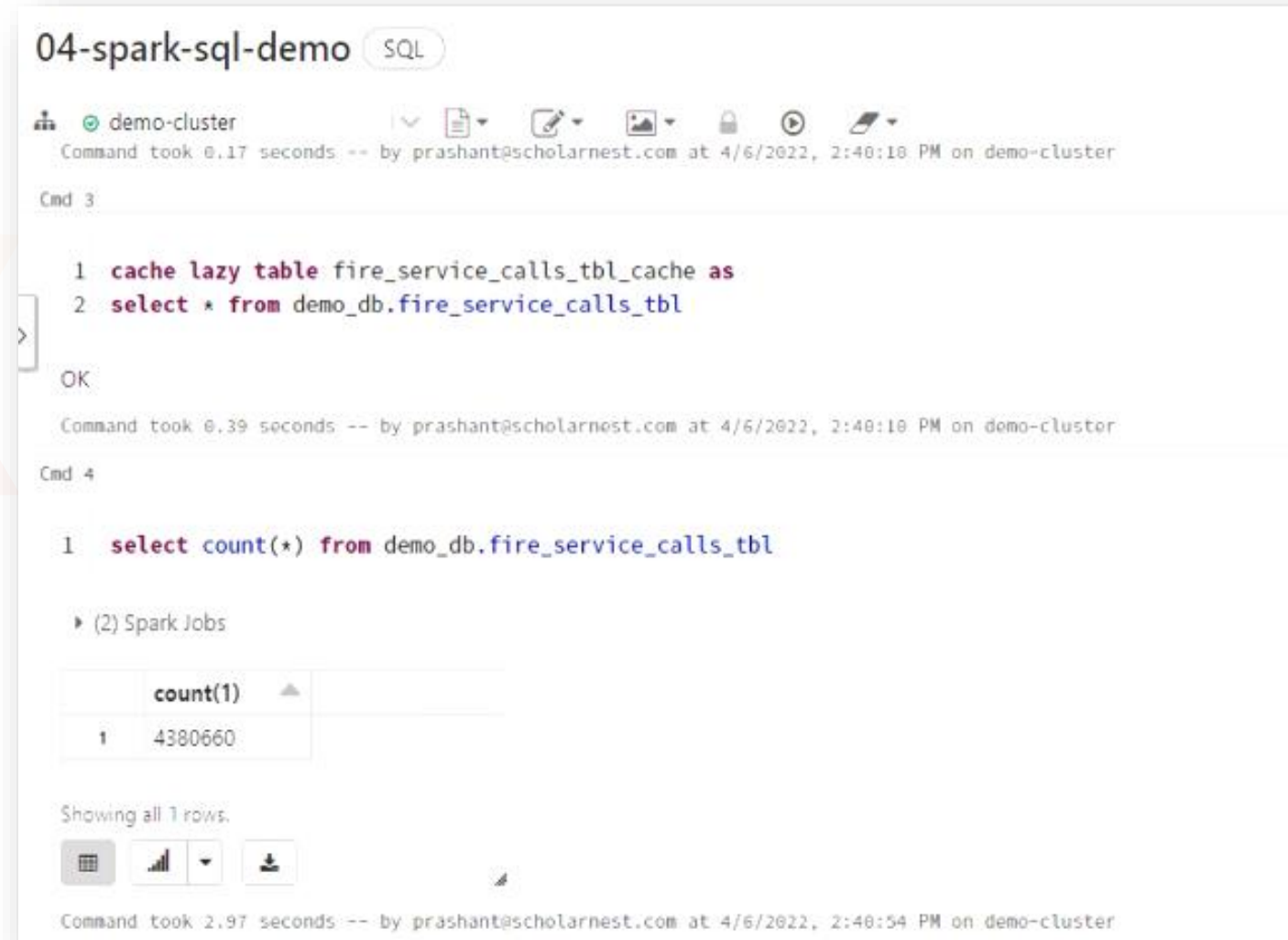
```
1 drop view if exists fire_service_calls_tbl_cache;
```

OK

Command took 0.17 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:40:10 PM on demo-cluster

Cmd 3

Let us start by taking a simple count from the table. We can use the select count(*) query. If we run it, we can see there are more than 4.3 million records in the table.



The screenshot shows a Databricks SQL interface with two commands executed on a cluster named 'demo-cluster'.

Cmd 3:

```
1 cache lazy table fire_service_calls_tbl_cache as
2 select * from demo_db.fire_service_calls_tbl
```

Command took 0.17 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:40:10 PM on demo-cluster

Cmd 4:

```
1 select count(*) from demo_db.fire_service_calls_tbl
```

Command took 0.39 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:40:10 PM on demo-cluster

(2) Spark Jobs

	count(1)
1	4380660

Showing all 1 rows.

Command took 2.97 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:40:54 PM on demo-cluster

Here are ten questions we want to answer:

1. How many distinct types of calls were made to the fire department?
2. What are distinct types of calls made to the fire department?
3. Find out all responses or delayed times greater than 5 mins?
4. What were the most common call types?
5. What zip codes accounted for the most common calls?
6. What San Francisco neighbourhoods are in the zip codes 94102 and 94103
7. What was the sum of all calls, average, min, and max of the call response times?
8. How many distinct years of data are in the CSV file?
9. What week of the year in 2018 had the most fire calls?
10. What neighbourhoods in San Francisco had the worst response time in 2018?

Databricks Notebook allows us to document things using markdown language.

You can use the %md command to create a documentation cell, as shown in the image given below.



The screenshot displays a Databricks Notebook interface. At the top, the notebook is titled "04-spark-sql-demo" with a "SQL" tab selected. Below the title, the "demo-cluster" is active. A table with one row is shown, containing the value "4380660" under the column "count(1)". The text "Showing all 1 rows." is displayed below the table. A toolbar with various icons is visible above the table. Below the table, a status bar indicates "Command took 2.97 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:40:54 PM on demo-cluster". The main area of the notebook shows a code cell labeled "Cmd 5" with two lines of text: "%md" and "#### Q1. How many distinct types of calls were made to the Fire Department?". The cell is in "Markdown" mode, as indicated by the "Markdown" button in the top right corner of the cell. A "Shift+Enter to run" prompt is visible at the bottom left of the notebook area.

count(1)
4380660

Showing all 1 rows.

Command took 2.97 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:40:54 PM on demo-cluster

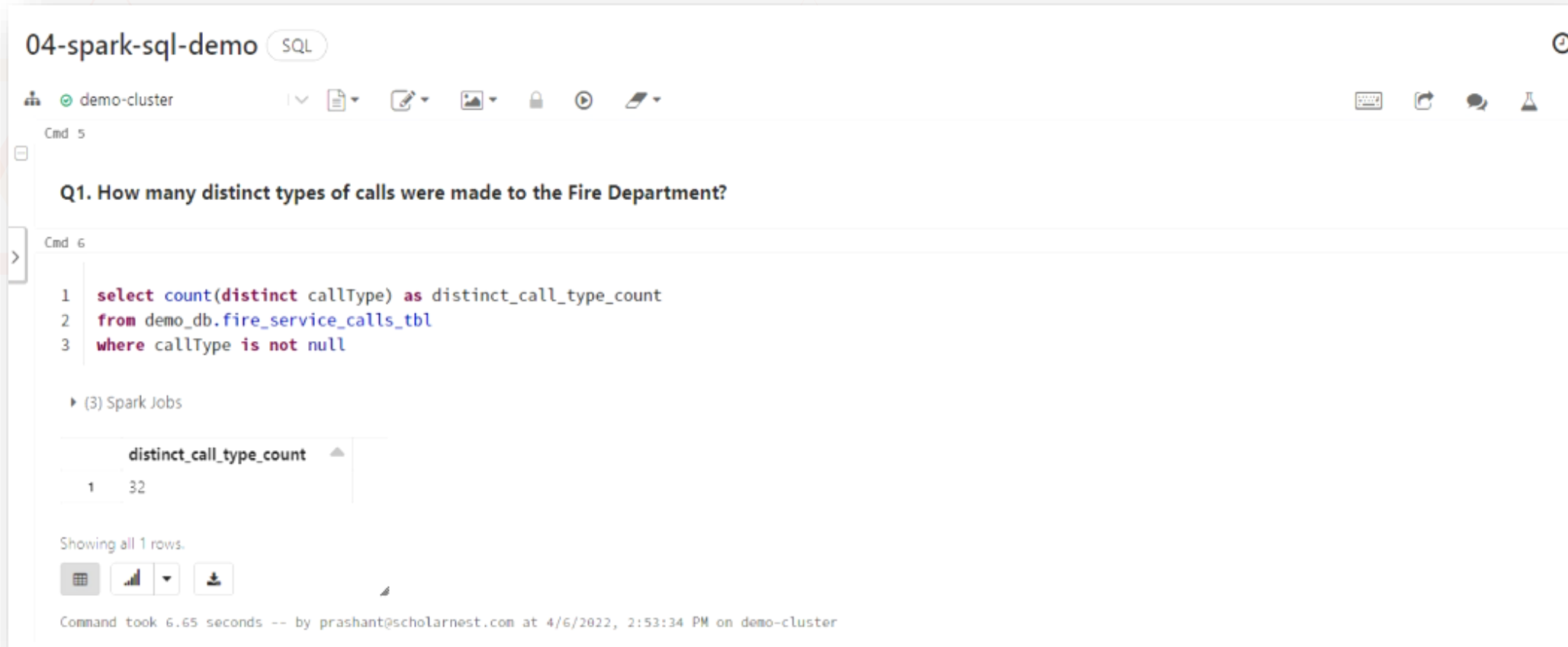
```
Cmd 5
1 %md
2 #### Q1. How many distinct types of calls were made to the Fire Department?
```

Shift+Enter to run

Let us look at the first question.

They are asking for how many distinct types of calls were made. So the solution is to take count of distinct call types. And we have the SQL code which says *select distinct call type* and then take a *count*. I added one extra condition to eliminate null values in the cell type.

And we got the expected output below the cell. We have 32 unique call types made to the San Francisco Fire station.



The screenshot shows a Databricks SQL interface with the title "04-spark-sql-demo". The interface includes a toolbar with icons for file operations, a cluster selector showing "demo-cluster", and a command input area. The command input area contains the following SQL query:

```
Cmd 5  
Q1. How many distinct types of calls were made to the Fire Department?  
Cmd 6  
1 select count(distinct callType) as distinct_call_type_count  
2 from demo_db.fire_service_calls_tbl  
3 where callType is not null
```

Below the query, it indicates "(3) Spark Jobs" and displays the result in a table:

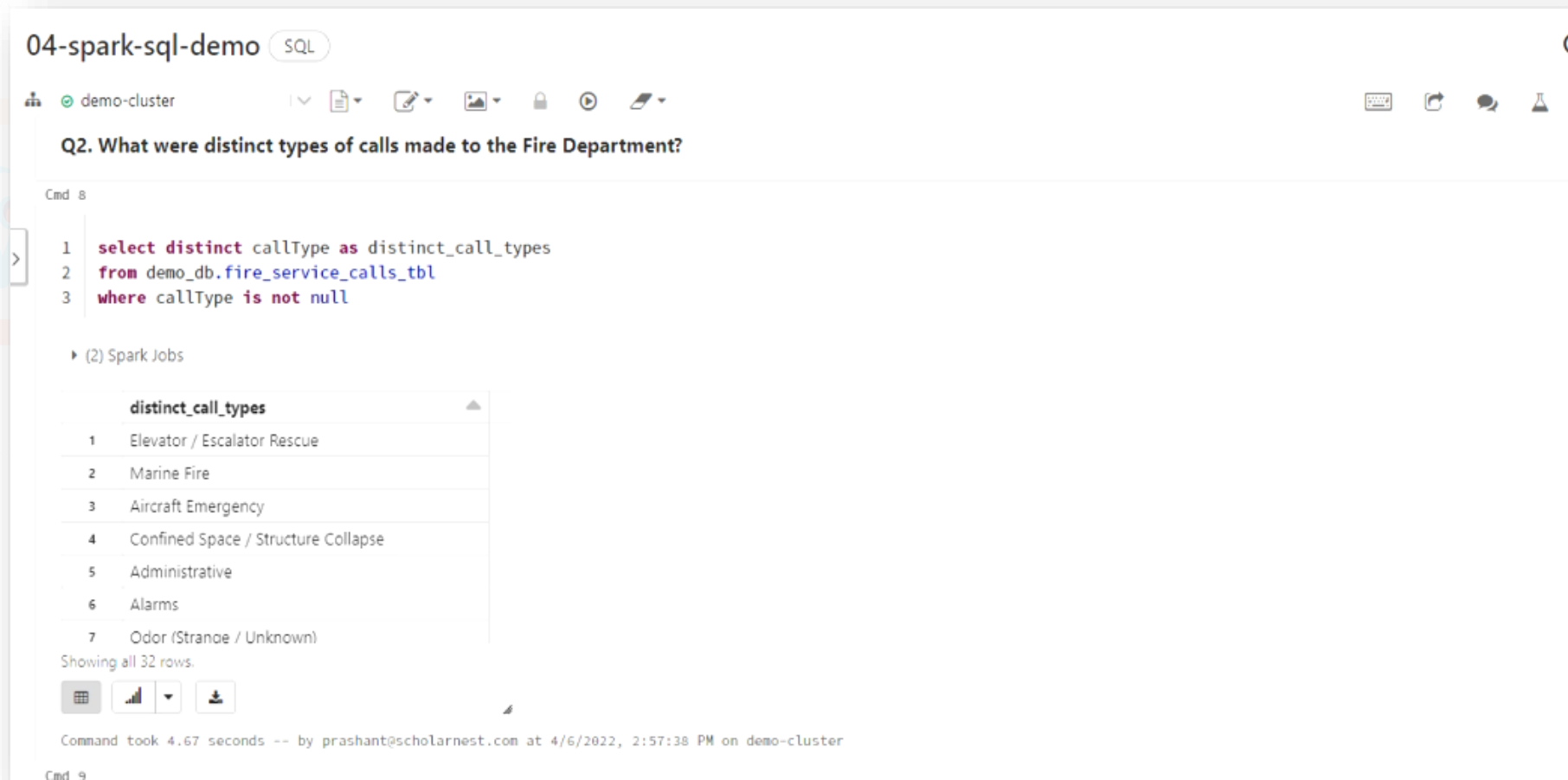
	distinct_call_type_count
1	32

Below the table, it says "Showing all 1 rows." and provides icons for table, chart, and download. At the bottom, a status bar indicates: "Command took 6.65 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:53:34 PM on demo-cluster".

Let us look at the second question.

So the question is almost the same as the first one, but this time, we need a list of distinct call types. Earlier, we wanted to count distinct call types, but now we want the list.

We have the code for the same shown below. I removed the count function from the select expression we used in the first question, and we can see the expected results.



The screenshot shows a Databricks SQL interface with the following components:

- Header:** "04-spark-sql-demo" with a "SQL" tab.
- Cluster:** "demo-cluster" with various status icons.
- Question:** "Q2. What were distinct types of calls made to the Fire Department?"
- Command:** "Cmd 8"
- SQL Query:**

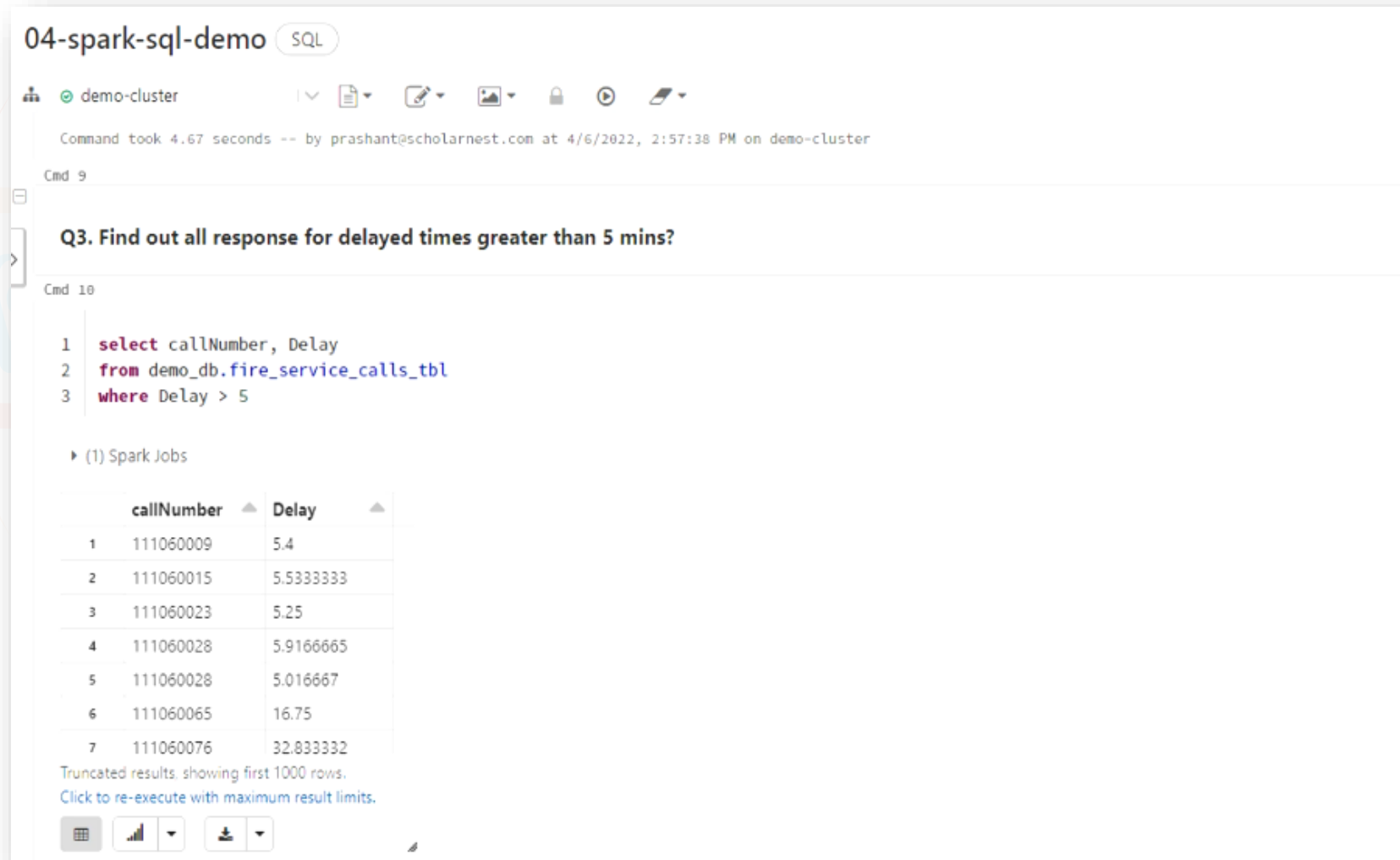
```
1 select distinct callType as distinct_call_types
2 from demo_db.fire_service_calls_tbl
3 where callType is not null
```
- Execution:** "(2) Spark Jobs" with a progress bar.
- Results Table:**

	distinct_call_types
1	Elevator / Escalator Rescue
2	Marine Fire
3	Aircraft Emergency
4	Confined Space / Structure Collapse
5	Administrative
6	Alarms
7	Odor (Strand / Unknown)
- Footer:** "Showing all 32 rows." and "Command took 4.67 seconds -- by prashant@scholarnest.com at 4/6/2022, 2:57:38 PM on demo-cluster".

Let us look at the third question.

We want to list the call numbers where the delay is greater than 5 minutes.

We have the code and desired results shown below.



The screenshot shows a Databricks SQL interface for a workspace named '04-spark-sql-demo'. The command executed was 'Q3. Find out all response for delayed times greater than 5 mins?'. The SQL query is as follows:

```
1 select callNumber, Delay
2 from demo_db.fire_service_calls_tbl
3 where Delay > 5
```

The results are displayed in a table with 7 rows, showing call numbers and their corresponding delays. The table is titled '(1) Spark Jobs'.

	callNumber	Delay
1	111060009	5.4
2	111060015	5.5333333
3	111060023	5.25
4	111060028	5.9166665
5	111060028	5.016667
6	111060065	16.75
7	111060076	32.833332

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Let us look at the fourth question.

We have to find the most common call types. So, we need to take a count on the call type and sort the result in descending order, and we will get the most common calls at the top. I have written the code for the same and you can see the most common types in sorted order.

04-spark-sql-demo SQL

demo-cluster

Cmd 11

Q4. What were the most common call types?

Cmd 12

```
1 select callType, count(*) as count
2 from demo_db.fire_service_calls_tbl
3 where callType is not null
4 group by callType
5 order by count desc
```

▶ (2) Spark Jobs

	callType	count
1	Medical Incident	2843475
2	Structure Fire	578998
3	Alarms	483518
4	Traffic Collision	175507
5	Citizen Assist / Service Call	65360
6	Other	56961
7	Outside Fire	51603

Showing all 32 rows.

Command took 5.93 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:03:57 PM on demo-cluster

Let us look at the fifth question.

Here we want to find the most common call types, but we also want to see the zip code for the most common call types. I can do that by adding a zip code column in my group by clause.

04-spark-sql-demo SQL

demo-cluster

Cmd 13

Q5. What zip codes accounted for most common calls?

Cmd 14

```
1 select callType, zipCode, count(*) as count
2 from demo_db.fire_service_calls_tbl
3 where callType is not null
4 group by callType, zipCode
5 order by count desc
```

▶ (2) Spark Jobs

	callType	zipCode	count
1	Medical Incident	94102	401457
2	Medical Incident	94103	370215
3	Medical Incident	94110	249279
4	Medical Incident	94109	238087
5	Medical Incident	94124	147564
6	Medical Incident	94112	139565
7	Medical Incident	94115	120087

Showing all 714 rows.

Command took 5.64 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:09:33 PM on demo-cluster

Let us look at the sixth question.

This question wants us to filter the records for two zip codes - 94102 and 94103. Then we want to see the names of unique neighbourhoods in these zip codes.

04-spark-sql-demo SQL

demo-cluster

Q6. What San Francisco neighborhoods are in the zip codes 94102 and 94103?

Cmd 16

```
1 select zipCode, neighborhood
2 from demo_db.fire_service_calls_tbl
3 where zipCode == 94102 or zipCode == 94103
```

▶ (1) Spark Jobs

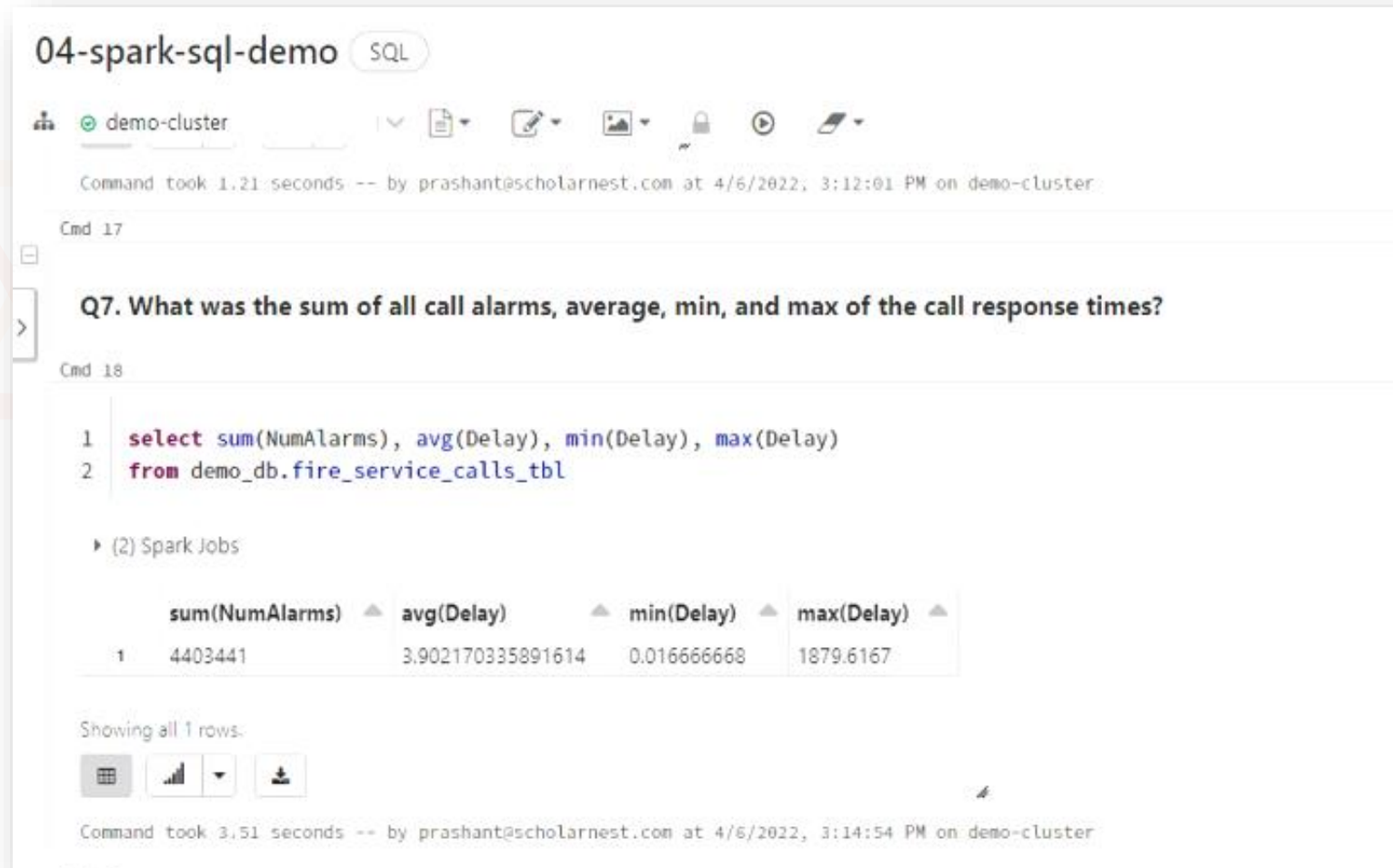
	zipCode	neighborhood
1	94103	South of Market
2	94103	South of Market
3	94102	Tenderloin
4	94102	Tenderloin
5	94102	Tenderloin
6	94102	Tenderloin
7	94102	Tenderloin

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 1.21 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:12:01 PM on demo-cluster

Let us look at the seventh question.

The first part of the question wants us to find the sum of *NumAlarms*. The second part of the question wants us to find avg, min, and max response time delay. And we want to find it for all the calls. So no filter and no grouping.



The screenshot displays a Databricks SQL interface for a workspace named '04-spark-sql-demo'. The interface shows a command execution history with 'Cmd 17' and 'Cmd 18'. 'Cmd 18' contains a SQL query to calculate the sum of alarms and average, minimum, and maximum delays from a table named 'demo_db.fire_service_calls_tbl'. The query results are shown in a table with one row of data. Below the table, it indicates 'Showing all 1 rows.' and provides icons for table, chart, and download. The command execution time is noted as 3.51 seconds.

04-spark-sql-demo SQL

demo-cluster

Command took 1.21 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:12:01 PM on demo-cluster

Cmd 17

Q7. What was the sum of all call alarms, average, min, and max of the call response times?

Cmd 18

```
1 select sum(NumAlarms), avg(Delay), min(Delay), max(Delay)
2 from demo_db.fire_service_calls_tbl
```

▶ (2) Spark Jobs

	sum(NumAlarms)	avg(Delay)	min(Delay)	max(Delay)
1	4403441	3.902170335891614	0.016666668	1879.6167

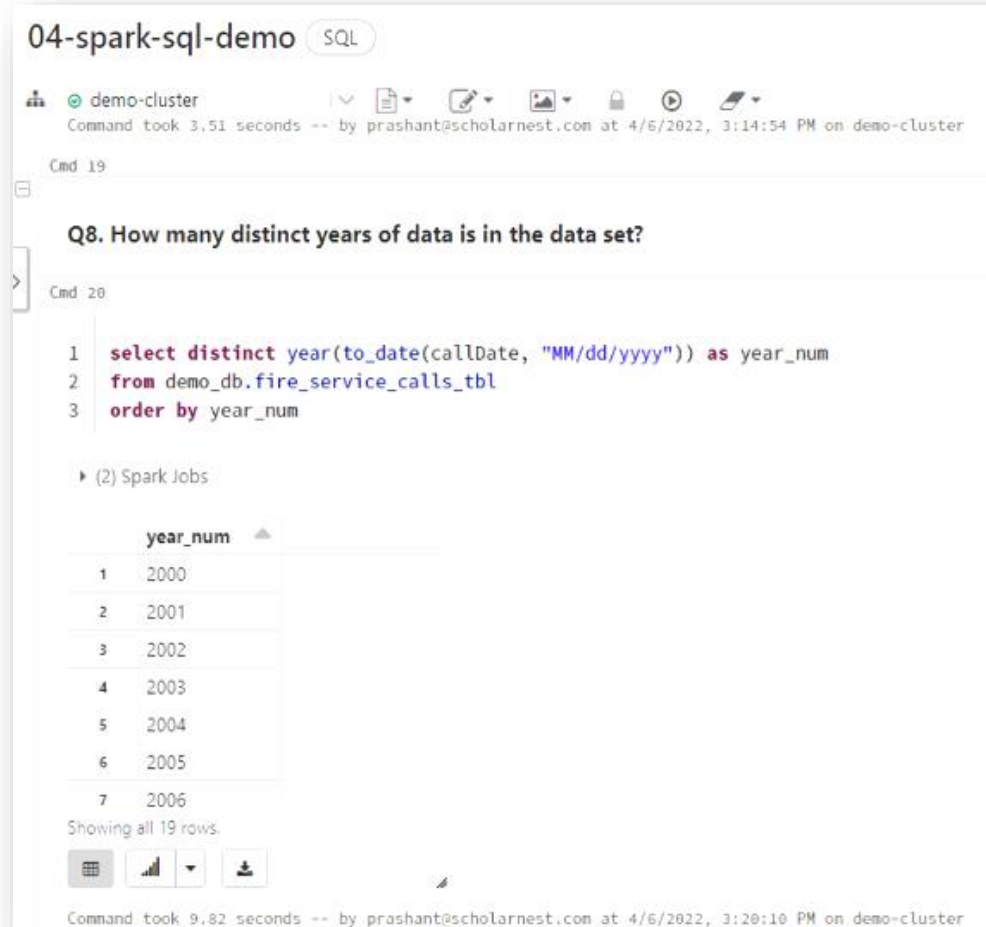
Showing all 1 rows.

Command took 3.51 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:14:54 PM on demo-cluster

Let us look at the eighth question.

They want us to find the number of distinct years in the data set. We have a Call Date field, and we can take out the year from that field. However, the Call Date is a string field.

But we can convert it to a date field and take out the year. Once we have the year number, we can apply a distinct clause to find unique years.



04-spark-sql-demo SQL

demo-cluster
Command took 3.51 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:14:54 PM on demo-cluster

Cmd 19

Q8. How many distinct years of data is in the data set?

Cmd 20

```
1 select distinct year(to_date(callDate, "MM/dd/yyyy")) as year_num
2 from demo_db.fire_service_calls_tbl
3 order by year_num
```

(2) Spark Jobs

	year_num
1	2000
2	2001
3	2002
4	2003
5	2004
6	2005
7	2006

Showing all 19 rows.

Command took 9.82 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:20:10 PM on demo-cluster

Let us look at the ninth question.

They want us to filter the table for the year 2018. We can convert the CallDate into a date field, take out the year and filter it for 2018. Then we can take out the week number from the CallDate and count it by week number. Once counted, you can sort it in descending order to find the most calls.

04-spark-sql-demo SQL

demo-cluster
Cmd 21

Q9. What week of the year in 2018 had the most fire calls?

Cmd 22

```
1 select weekofyear(to_date(callDate, "MM/dd/yyyy")) week_year, count(*) as count
2 from demo_db.fire_service_calls_tbl
3 where year(to_date(callDate, "MM/dd/yyyy")) == 2018
4 group by week_year
5 order by count desc
```

▶ (2) Spark Jobs

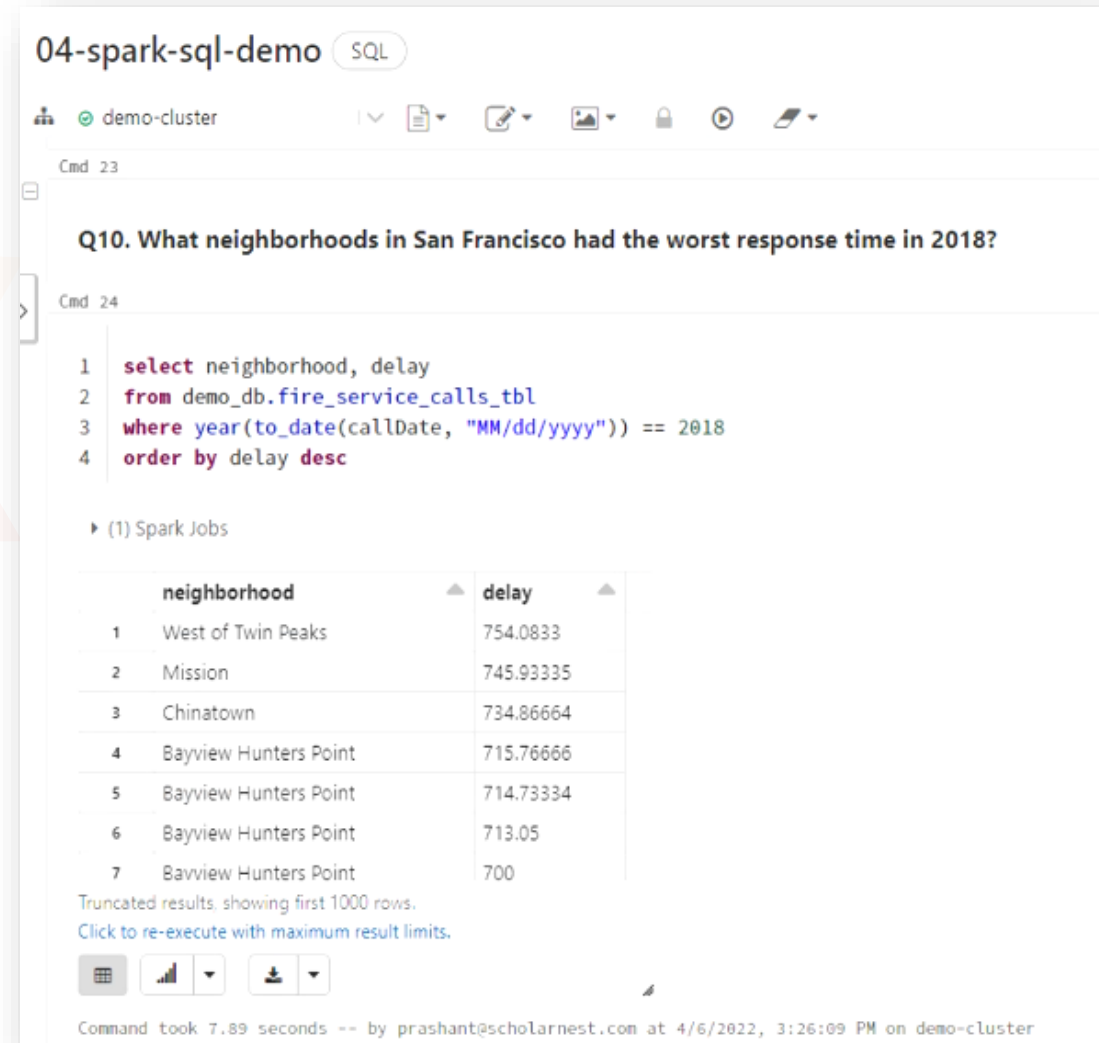
	week_year	count
1	1	6401
2	25	6163
3	13	6103
4	22	6060
5	44	6048
6	27	6042
7	16	6009

Showing all 45 rows.

Command took 8.23 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:24:15 PM on demo-cluster

Let us look at the tenth question.

We just need to filter records for the year 2018. Then look for the neighbourhoods and delay columns. And finally, sort it by delay in descending order.



The screenshot shows a Databricks SQL interface with a query editor and a results table. The query is for Q10, asking for neighborhoods in San Francisco with the worst response time in 2018. The results table shows 7 rows of data, sorted by delay in descending order.

04-spark-sql-demo SQL

demo-cluster

Cmd 23

Q10. What neighborhoods in San Francisco had the worst response time in 2018?

Cmd 24

```
1 select neighborhood, delay
2 from demo_db.fire_service_calls_tbl
3 where year(to_date(callDate, "MM/dd/yyyy")) == 2018
4 order by delay desc
```

▶ (1) Spark Jobs

	neighborhood	delay
1	West of Twin Peaks	754.0833
2	Mission	745.93335
3	Chinatown	734.86664
4	Bayview Hunters Point	715.76666
5	Bayview Hunters Point	714.73334
6	Bayview Hunters Point	713.05
7	Bayview Hunters Point	700

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 7.89 seconds -- by prashant@scholarnest.com at 4/6/2022, 3:26:09 PM on demo-cluster



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com